

Exercice 1. Dans toutes les questions suivantes, on suppose disposer en OCaml des structures `'a stack` et `'a queue` avec leurs opérations usuelles. On définit également un type `'a pair = {mutable first : 'a ; mutable second : 'a}`

1) On aimerait disposer d'une structure de données `min_stack` qui nous donne non seulement la possibilité de regarder l'élément au sommet de la pile, mais également la possibilité de regarder le plus petit élément de la pile. Décrire une implémentation possible et donner la complexité de ses opérations. (*Indication* : on se servira directement de la structure de `stack` sans se soucier de son implémentation).

2) Pour la structure précédente, écrire les fonctions `push`, `top` et `st_min` où les deux premières correspondent à l'équivalent pour un `stack` usuel et `st_min` renvoie le minimum de la pile.

Dans les questions suivantes, nous souhaitons décrire une structure analogue pour les files, mais la seule information dont on a besoin est le minimum des éléments présents dans la file (on suppose ne plus avoir besoin de la valeur du sommet de la file).

3) Expliquer pourquoi effectuer la même modification aux files que celle qu'on a effectué aux piles en question 1 ne fonctionnerait pas. On pourra s'aider d'un contre-exemple.

4) Décrire succinctement une implémentation possible de double file `'a deque` et donner les complexités de ses opérations.

5) Redéfinir une fonction `push_back` de signature `'a -> 'a deque -> unit` qui ajoute un élément à une double file en conservant toujours l'invariant suivant : "Tous les éléments de la double file sont dans un ordre croissant", c'est-à-dire que si l'on appelle un nombre arbitraire de fois `pop_front` alors les résultats successifs sont croissants. On suppose ici que l'on peut se débarrasser de n'importe quel élément de la double file si on le souhaite.

6) Décrire alors comment implémenter une structure de `min_queue` en écrivant ses fonctions `push` et `pop`. *Indication* : on pourra s'aider de la question 5 et donner un indice i à chaque élément que l'on insère dans la file).

7) Expliquer comment nous pouvons en fait utiliser deux `min_stack` pour implémenter une `min_queue` qui nous donne en plus l'information de l'élément au sommet.

Exercice 2 : Sommes d'intervalles. Dans chacune des questions suivantes, il est demandé d'écrire le code de l'algorithme demandé dans le langage voulu : C ou OCaml. Soit T un tableau d'entiers relatifs de taille N .

1) Donner un algorithme simple pour calculer des indices i et j tels que $0 \leq i \leq j < N$ et tels que la somme $\sum_{k=i}^j T[k]$ soit maximale. Quelle est sa complexité en temps ?

2) Si, pour une valeur de j donnée, on connaît la valeur de $\max_{i \in [0, j]} \sum_{k=i}^j T[k]$, comment calculer efficacement $\max_{i \in [0, j+1]} \sum_{k=i}^{j+1} T[k]$? En déduire un algorithme en temps linéaire pour répondre à la question 1.

3) Soit $m \in [1, N]$. Donner un algorithme pour calculer en temps linéaire un indice $i \in [0, N - m]$ tel que la somme $\sum_{k=i}^{i+m-1} T[k]$ soit maximale.

4) Proposer un algorithme pour répondre à la question 1 en ajoutant la contrainte $j - i < m$

Exercice 3 : Algorithme Shunting-Yard. Soit $Arith$ l'ensemble des chaînes de caractères constituées des caractères dans `"0123456789+-*/()` et qui correspondent à des expressions arithmétiques valides. Par exemple, `"13*5+1-6/3"` $\in Arith$, tandis que `"0++2"` $\notin Arith$. Nous supposons disposer de `int comp(char* a, char* b)` qui renvoie -1 si l'opérateur a est moins prioritaire que b , 0 s'il l'est autant, et 1 sinon. Par exemple, `comp('+', '*')` renvoie -1, tandis que `comp('+', '-')` renvoie 0. La fonction n'est pas définie lorsqu'une des deux chaînes en entrée représente un nombre.

1) Définir un type `Tree` en C qui représente un arbre binaire de chaînes de caractères.

On suppose disposer d'un type `str_list` qui représente une liste chaînée de `char*` et dont les champs s'appellent `cur` et `next`. Cette liste nous permet de représenter des expressions de la façon suivante : `(1 + 33) * 2` sera représenté par `[("(", "1", "+", "33", ")"), "*", "2"]`.

2) Décrire succinctement deux façons différentes d'implémenter une structure `stack_tree` qui représente une pile d'arbres.

On suppose par la suite disposer également d'une structure `stack_str` qui représente une pile de chaînes de caractères.

3) Écrire une fonction `Tree* str_to_tree(str_list* lst)` qui transforme une expression arithmétique en son arbre correspondant. On utilisera pour cela le principe de la gare de triage : on garde une pile d'opérateurs et une pile d'arbres en mémoire et on construit l'arbre progressivement en retirant des opérateurs de la première pile quand il le faut. *Indication : l'arbre ne demandé est tel que les noeuds internes prennent des valeurs dans $\{+, -, *, /\}$ et les feuilles représentent des nombres. On pourra commencer par dessiner les arbres correspondants aux exemples au-dessus.*

4) Écrire une fonction `int eval(Tree* t)` qui évalue un arbre correspondant à une expression arithmétique. On rappelle que la fonction `int atoi(char* s)` de la librairie standard renvoie le nombre correspondant à une chaîne.

5) Expliquer succinctement comment reconstruire l'expression arithmétique initiale à partir de l'arbre, quitte à rajouter des parenthèses.

6) Donner les arbres correspondant à $13 * 5 + 1 - 6/3$ et à $(1 + 33) * 2$ ainsi que l'ordre de leurs noeuds lors de leur parcours suffixe.

7) Expliquer comment on peut évaluer une expression arithmétique directement à partir de l'ordre du parcours suffixe de son arbre.

Note : On remarque alors que cette nouvelle façon d'écrire des expressions ne nécessite aucune parenthèse. On l'appelle notation polonaise inverse et c'est celle qui fut utilisée dans les premières calculatrices HP

Exercice 4. On définit la suite des Arbres binomiaux $(B_k)_{k \in \mathbb{N}}$ (non étiquetés) de la façon suivante : B_0 a un seul noeud et $\forall k \geq 1, B_k$ est constitué d'une racine ayant pour fils chacun des arbres $B_0, \dots, B_{k-1}$.

- 1) Dessiner B_4
- 2) Quels sont la taille, la hauteur ainsi que le nombre de noeuds internes de B_n ?
- 3) Donner une définition possible de 'a arbre pour contenir un arbre d'arité quelconque.
- 4) Écrire une fonction `binom_tree` qui prend un entier n et renvoie B_n . Donner sa complexité.
- 5) Écrire une fonction `est_binomial` qui renvoie un booléen indiquant si l'arbre passé en argument est binomial.

On suppose maintenant que l'on étiquette B_k par l'écriture en binaire de l'ordre de chaque noeud lors du parcours suffixe de l'arbre. On suppose pour cela que les enfants de la racine de B_k sont dans l'ordre suivant : $B_{k-1}, B_{k-1}, \dots, B_0$

- 6) Dessiner alors B_4 étiqueté.
- 7) On considère un noeud n d'étiquette l à la profondeur i , et on considère $j = k - i$. Montrer que le nombre de 1 que contient x dans sa représentation binaire est égal à j .
- 8) Montrer que le degré de n est égal au nombre de 1 situés à droite du 0 le plus à droite dans la représentation binaire de l .

Exercice 5. On représente des arbres non étiquetés par $Nil \mid N(e, lst)$. On dit que deux arbres non étiquetés A et B sont isomorphes s'ils vérifient l'une des propriétés suivantes: $A = B = Nil$, ou $A = N(e, l_1), B = N(e, l_2)$ où il existe une bijection ϕ entre les éléments de l_1 et l_2 telle que $\forall t \in l_1, t$ est isomorphe à $\phi(t)$. Autrement dit, A et B sont identiques à réordonnement des fils de chaque noeud près. On dit qu'on enracine un arbre en un noeud quelconque s lorsqu'on transforme l'arbre pour mettre s en racine. Formellement, si l'on appelle enracine notre opération, il s'agit de mettre s à la racine et de lui donner comme enfants ses enfants actuel en plus de **enracine** $(p, A \setminus \{s\})$ où p est le parent de s et $A \setminus \{s\}$ est l'arbre actuel privé de s ainsi que de ses enfants.

- 1) Dessiner les différentes façons d'enraciner $N(e, [N(e, [N(e, []), N(e, [])])])$
- 2) Montrer que pour $n \geq 7$, il existe T_n un arbre de taille n tel que tous les arbres donnés par les différents enracinements de T_n ne sont pas isomorphes deux à deux.

Exercice 6. Soit $\mathcal{A}$ l'ensemble des arbres. Pour tout noeud n d'un arbre $A \in \mathcal{A}$, on définit son **degré** par le nombre de noeuds auquel il est lié. En clair, le degré de n vaut son arité +1 si n a un parent, et son arité sinon.

Soit $A \in \mathcal{A}$ un arbre. Alice et Bob jouent au jeu suivant à tour de rôle : soit F l'ensemble des noeuds de degré 1 de A . Le joueur actuel choisit un noeud $f \in F$ et le retire de A . Si f est la racine de A , alors le joueur actuel gagne. On suppose par la suite qu'Alice joue toujours le premier coup.

1) Montrer que A reste bien un arbre pendant la durée du jeu et qu'il y a toujours un gagnant à la fin du jeu.

On admet que pour A donné, soit Alice possède une stratégie gagnante, c'est-à-dire une stratégie qui lui permet de gagner peu importe les coups de Bob, soit Bob possède une stratégie gagnante.

2) Caractériser l'ensemble gagnant $G(\text{Alice}) \subset \mathcal{A}$, c'est-à-dire l'ensemble des arbres tels qu'Alice possède une stratégie gagnante pour le jeu sur cet arbre.

Exercice 7. Soit $\mathcal{A}_B$ l'ensemble des arbres binaires définis en OCaml par :

```
type arbre = E | N of arbre * int * arbre
```

1

On dit qu'un arbre A est un tas binaire-min si $A = E$ ou bien $A = N(g, x, d)$ où g et d sont des tas binaires-min et x est inférieur ou égal à tous les éléments de g et d .

1) Écrire une fonction **est_tas** de signature **arbre** -> **bool** qui prend un arbre en argument et renvoie si c'est un tas. On garantira une complexité $O(|A|)$.

2) Montrer qu'il y a exactement $n!$ arbres croissants possédant n noeuds étiquetés par les entiers $1, \dots, n$ (chaque noeud étant étiqueté par un entier distinct).

3) On définit le *potentiel* d'un tas binaire-min A , noté $\Phi(A)$, par son nombre de noeuds $N(g, x, d)$ tels que $|g| < |d|$. Écrire une fonction **potentiel** qui renvoie $\Phi(A)$. On garantira une complexité $O(|A|)$.

On définit la fonction **fusion** par induction de la façon suivante :

$$\begin{aligned} \text{fusion}(A_1, E) &= A_1 \\ \text{fusion}(E, A_2) &= A_2 \\ \text{fusion}(N(g_1, x_1, d_1), N(g_2, x_2, d_2)) &= N(\text{fusion}(d_1, N(g_2, x_2, d_2)), x_1, g_1) \text{ si } x_1 \leq x_2 \\ \text{fusion}(N(g_1, x_1, d_1), N(g_2, x_2, d_2)) &= N(\text{fusion}(d_2, N(g_1, x_1, d_1)), x_2, g_2) \text{ sinon} \end{aligned}$$

4) Dessiner **fusion**($N(N(E, 2, E), 1, N(E, 4, E)), N(N(E, 5, E), 3, N(E, 6, E))$)

5) Soit $C(A_1, A_2)$ le *coût* de la fusion des tas binaires-min A_1 et A_2 , c'est-à-dire le nombre d'appels récursifs lors de l'exécution de **fusion**(A_1, A_2). Soient A_1 et A_2 deux tas binaires-min et A le résultat de **fusion**(A_1, A_2), montrer que

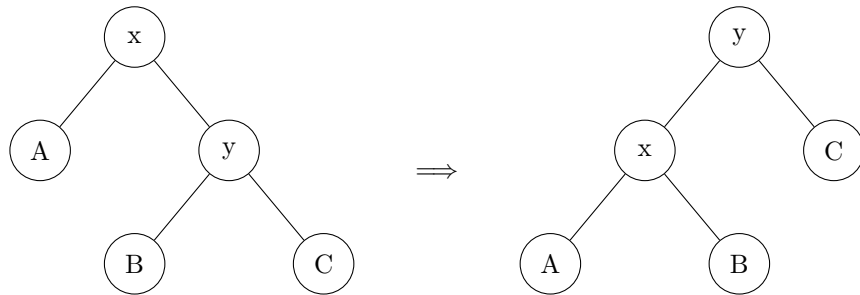
$$C(A_1, A_2) \leq \Phi(A_1) + \Phi(A_2) - \Phi(A) + 2(\log|A_1| + \log|A_2|)$$

Note : ce résultat central permet alors de montrer que l'on peut alors trier des tableaux à l'aide de nos tas binaires en complexité $O(\log n)$

Dans toute la suite, on considère des arbres binaires de recherche définis en OCaml par le type suivant :

```
type 'a bst = Nil | Node of 'a bst * 'a * 'a bst
```

Exercice 8. On rappelle le principe d'une rotation gauche dans un ABR :



1) Écrire une fonction **rotation_gauche** qui implémente l'opération ci-dessus

2) Montrer qu'un arbre binaire de recherche à n noeuds peut être transformé en n'importe quel autre arbre de recherche comportant les mêmes noeuds à l'aide de $O(n)$ rotations (gauches ou droites).

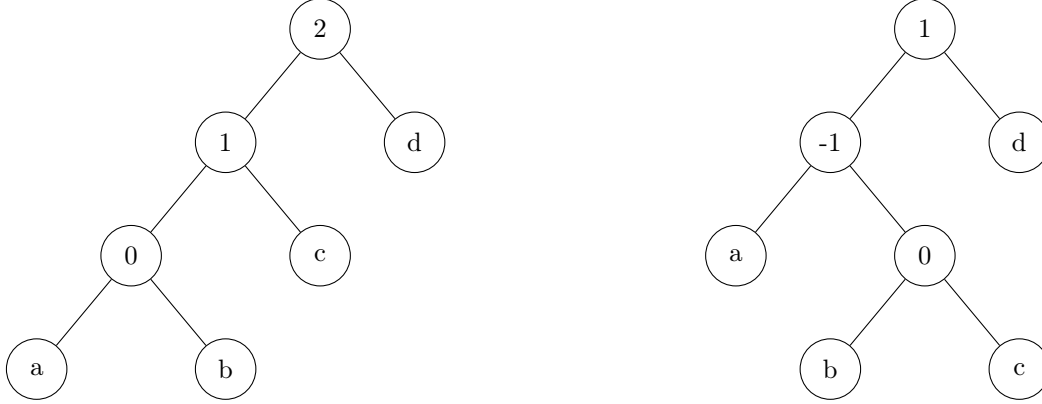
Exercice 9.

1) D  duire de l'exercice pr  c  dent un algorithme qui prend un arbre binaire de recherche et l'  tiquette de l'un de ses n  uds et transforme cet arbre binaire de recherche en d  pla  ant ce n  ud    la racine. Quelle est la complexit   en temps de cet algorithme?

2) Proposer une am  lioration de l'algorithme de recherche dans un arbre binaire de recherche, qui r  pond plus rapidement aux requ  tes de recherche qui lui sont pos  es fr  quemment. On ne demande pas ici de preuve de complexit  .

Malheureusement, l'algorithme pr  sent      la question 3 n'a pas de bonnes propri  t  s de complexit   th  oriques. Nous allons   tudier une autre approche.

3) Consid  rer les ABR suivants :



Comment peut-on les transformer afin de placer le n  ud 0    la racine ? n d  duire une variante de l'algorithme donn   en Question 3. Est-elle   quivalente ?

On d  finit maintenant la taille d'un arbre a , not  e $\text{taille}(a)$, comme   tant le nombre de n  uds qu'il contient. Son rang $\text{rg}(a)$ est d  fini par $\text{rg}(a) = \log_2(\text{taille}(a))$. Enfin, le potentiel $\Phi(a)$ d'un arbre a est la somme des rangs de tous ses sous-arbres.

4) Soit t un arbre, et soit t' l'arbre r  sultant d'une rotation dans t (pas n  cessairement    sa racine). De plus, soit t'_0 le sous-arbre de t' sur lequel la rotation a   t   effectu  e, et t_0 le sous-arbre de t dont la racine a la m  me   tiquette que celle de t'_0 . Montrer que :

$$\Phi(t') - \Phi(t) \leq 3(\text{rg}(t'_0) - \text{rg}(t_0))$$

5) Dans cette question, consid  rons les diff  rentes transformations d  crites en 3) plut  t qu'une rotation. Montrer que :

$$\Phi(t') - \Phi(t) \leq 3(\text{rg}(t'_0) - \text{rg}(t_0)) - k$$

O   k est une constante enti  re qui d  pend de la transformation que l'on devra calculer. On pourra utiliser, en l'admettant, l'in  galit   arithm  tico-g  om  trique.

Exercice 10 : Matro  des.

D  finition 0.0.1. Soit E un ensemble fini non vide et $I \subset \mathcal{P}(E)$. Le couple (E, I) est un matro  de lorsqu'il v  rifie les propri  t  s suivantes :

1. E est non-vide
2. Si $I_1 \in \mathcal{I}$ et $I_0 \subset I_1$ alors $I_0 \in \mathcal{I}$
3. Si $(I_0, I_1) \in \mathcal{I}^2$ et $|I_0| < |I_1|$ alors $\exists e \in I_1 \setminus I_0$ tel que $I_0 \cup \{e\} \in \mathcal{I}$

Les   l  ments de $\mathcal{I}$ sont alors appel  s *ensembles ind  pendants* pour ce matro  de.

1) Soit $G = (S, A)$ un graphe non orienté. Montrer que $M_G = (E_G, \mathcal{I}_G)$ où $E_G = A$ et $F \in \mathcal{I}_G$ si et seulement si F n'induit aucun cycle dans G est bien un matroïde.

2) Soit $M = (E, \mathcal{I})$ un matroïde. Nous appelons un ensemble indépendant I *maximal* si $\forall x \in E \setminus I, I \cup \{x\} \notin \mathcal{I}$. Que peut-on dire sur le cardinal des différents ensembles maximaux de M ? Que peut-on dire dans le cas où $M = M_G$ où G est un graphe non orienté ?

3) M est dit pondéré lorsque l'on dispose d'une fonction w telle que $\forall x \in E, w(x) > 0$. Pour un tel matroïde, exhiber en s'inspirant de l'algorithme de Kruskal un algorithme calculant un ensemble indépendant de poids maximal dans M . Donner sa complexité en fonction de τ , le temps d'exécution de la fonction d'oracle indiquant si un ensemble est dans $\mathcal{I}$ et prouver sa correction.

4) Soient $M_1 = (E, \mathcal{I}_1)$ et $M_2 = (E, \mathcal{I}_2)$ deux matroïdes sur le même ensemble. Montrer que :

$$\max_{S \in \mathcal{I}_1 \cap \mathcal{I}_2} |S| \leq \min_{U \subset E} (r_1(U) + r_2(E \setminus U))$$

où les r_i sont les fonctions de *rang* associant à tout $X \subset E$ le cardinal de l'ensemble indépendant le plus grand contenu dans X . On admet par la suite que cette inégalité est en réalité une égalité. Il s'avère également qu'un tel ensemble S se calcule en temps polynomial en $|E|$.

5) Montrer le théorème de König : Si G est un graphe biparti connexe, alors la taille d'un couplage maximum de G est égale à la taille d'une couverture par les sommets minimum de G . (*Indice : on pourra se servir du matroïde constitué de tous les ensembles d'arêtes qui ne se rencontrent pas dans une des deux parties du graphe*).

Exercice 11 : Problème du postier.

Dans tout cet exercice, nous considérerons $G = (V, E)$ un graphe non orienté pondéré à poids strictement positifs. Notre but est de trouver un circuit (un chemin revenant à son origine) $W = e_1 \dots e_k$ qui est de longueur minimale et contenant toutes les arêtes de G .

1) Montrer qu'il existe une solution si et seulement si le graphe est connexe.

2) On dit de G qu'il est **Eulérien** s'il existe un circuit passant exactement une fois par chacune de ses arêtes. Montrer que si G est Eulérien alors chacun de ses sommets est de degré pair.

3) On suppose que G ne possède que des sommets de degré pair. Montrer que tout chemin de u à $v \in V$ passant au plus une fois par chaque arête et contenant toutes les arêtes connectées à v vérifie $u = v$ (i.e. est un circuit).

4) Montrer la réciproque de 2) (*Indication : on pourra construire un chemin de façon incrémentale*).

Nous avons ainsi montré que l'on peut construire facilement un chemin optimal lorsque cette condition est vérifiée. On se place maintenant dans le cas général. Nous admettons que les propriétés précédentes restent valides dans le cas des multi-graphes, c'est-à-dire des graphes où les arêtes entre deux sommets peuvent être multiples.

5) Nous nous donnons un chemin optimal W . Montrer que l'on peut décomposer les arêtes de W comme $E \cup F$, où E est l'ensemble des arêtes de G et F est un multi-ensemble à déterminer.

6) Montrer alors que le problème se réduit à optimiser la somme des poids des arêtes dans F .

7) Soit le multi-graphe $H = (V, E \cup F)$. Montrer que tous ses sommets sont de degré pair.

On pose G' le multi-graphe induit par F

8) Soit x le nombre de sommets de degré impair de G' . Montrer que x est pair.

9) Montrer que l'on peut créer une partition des arêtes de G' en $x/2$ chemins distincts, tels que les extrémités de ces chemins sont exactement les sommets de degré impair. (*Indication : On pourra commencer par construire quelque chose de plus gros qu'une partition, puis expliquer comment supprimer les cycles.*)

10) Montrer qu'alors le problème général se réduit à trouver un "appariement optimal" sur les sommets de degré impair de G .

Il se trouve que trouver un tel appariement est la partie la plus technique de l'algorithme, mais peut se faire grâce à l'algorithme Weighted Blossom en $O(n^2m)$. Le problème du postier, bien que difficile en pratique, est donc bien résoluble en temps polynomial

Exercice 12 : Détection de cycle. Pour chaque question nécessitant de décrire un algorithme, il est demandé de l'implémenter en C, en OCaml ou en pseudocode. Dans tous les cas, on suppose disposer d'une structure de dictionnaire pour lequel toutes les opérations se font en temps constant, et dont la complexité en mémoire est linéaire en son nombre d'entrées.

- 1) Soit $N > 2$ un entier et f de $\llbracket 0, N-1 \rrbracket$ dans lui-même. On définit $(x_n)_{n \in \mathbb{N}}$ par $x_0 < N$ et $\forall n \in \mathbb{N}, x_{n+1} = f(x_n)$. Montrer qu'il existe i et j tels que $x_i = x_j$ et $i < j$.
- 2) On pose $\mu = \min \{i \in \mathbb{N} \mid \exists j \in \mathbb{N}, j > i \wedge x_i = x_j\}$ et $\lambda = \min \{i \in \mathbb{N}^* \mid x_\mu = x_{\mu+i}\}$. Donner un algorithme efficace en temps pour calculer μ et λ . Quel est le nombre (exact) d'appels à f effectués ? Quelle est la complexité en mémoire, en fonction de λ , μ et N ? Le nombre d'appels à f est-il optimal ?
- 3) Donner maintenant un algorithme simple pour calculer μ et λ qui soit efficace en mémoire. Quelle est sa complexité en temps et en mémoire ?
- 4) On cherche maintenant un algorithme efficace en temps et en mémoire. Quels sont les éléments de l'ensemble $\{n \in \mathbb{N}^* \mid x_n = x_{2n}\}$ en fonction de λ et μ ?
- 5) En déduire un algorithme pour calculer λ et μ en temps $O(\lambda + \mu)$ et en mémoire $O(1)$.
- 6) Combien d'appels à f sont effectués au total par cet algorithme ?

Exercice 13 : Problème d'allumage de lampes.

On considère un réseau de lampes équipées d'interrupteurs que l'on modélise par un graphe orienté $G = (V, E)$. Chaque sommet $v \in V$ représente une lampe munie d'un interrupteur, et chaque arête orientée (a, b) représente le fait qu'appuyer sur l'interrupteur a change l'état de la lampe b (une lampe éteinte devient allumée et vice-versa). On s'intéresse au problème de savoir s'il est possible d'allumer chaque lampe lorsque toutes les lampes commencent éteintes.

- 1) Donner le graphe $G_{h,w}$ correspondant à la grille de hauteur h et de largeur w où l'interrupteur en (i, j) active toutes les lampes en (i', j') telles que $|i - i'| + |j - j'| \leq 1$

Nous allons désormais prouver le théorème suivant :

Théorème 1 : Soit $G = (V, E)$ tel que $\forall U \subset V$ avec $|U|$ impair, il existe un sommet de degré sortant impair dans le sous-graphe induit par U . Alors il est possible d'allumer toutes les lampes.

(On rappelle que le degré sortant d'un sommet est le nombre d'arêtes qui partent de ce sommet, et que le sous-graphe induit par U correspond au graphe dont les sommets sont U et les arêtes sont celles de G dont les extrémités sont incluses dans U)

- 2) Montrer que s'il existe une suite de pressions d'interrupteurs qui allume toutes les lampes, alors celle-ci peut être exprimée sous forme d'une partie $X \subset \mathcal{P}(V)$ (i.e. l'ordre ne compte pas et les redondances peuvent être simplifiées)

- 3) Nous allons montrer le théorème 1 par récurrence. Supposons que la propriété soit vérifiée au rang n . Démontrer l'hérédité dans le cas n impair. (*Indication : on pourra considérer la famille de sous-graphes $G \setminus \{v\}$ pour $v \in V$.*)

- 4) Démontrer l'hérédité dans le cas n pair (*Indication : adapter 3) en considérant un sommet de degré sortant impair*).

- 5) Que peut-on alors dire sur $G_{h,w}$?

Exercice 14 : Triangulation optimale.

Soit P un polygone convexe de sommets $A_1, \dots, A_n$. Une triangulation de P est un ensemble de diagonales ne s'intersectant pas qui partitionnent le polygone en triangles. Le but de cet exercice est de trouver une triangulation minimale, c'est-à-dire telle que la somme des longueurs de ses diagonales soit minimale.

- 1) Prouver que P admet au moins une triangulation.
- 2) On dit qu'une triangulation est "en zigzag" si elle est de la forme $\{A_i A_{i+2}, A_{i+2} A_{i-1}, A_{i-1} A_{i+3}, \dots\}$ où $1 \leq i \leq n$ et où les indices sont pris modulo n ($A_0 = A_n = A_{2n}$). Trouver un polygone tel qu'aucune triangulation optimale est en zigzag.

On admet que pour tous $1 \leq i \leq j \leq n$, $A_i A_{i+1} \dots A_j$ est encore un polygone convexe.

- 3) En se servant d'un tableau T en deux dimensions où l'on précisera la signification de $T[i][j]$, écrire en OCaml une fonction `triangulation_min : point array -> float` renvoyant la valeur d'une triangulation minimale de P . On supposera disposer d'une fonction `dist : point -> point -> float` et l'on pourra utiliser la fonction `make_matrix : int -> int -> 'a -> 'a array array`. (*Indication : pour trouver la solution au problème pour $A_i \dots A_j$, on utilisera uniquement des solutions du problème à des polygones inclus dans $A_i \dots A_j$*)

- 4) Quelle est la complexité de la fonction précédente ? Décrire brièvement comment on pourrait la modifier pour obtenir une triangulation minimale.

Exercice 15.

Les œufs se cassent lorsqu'ils sont lâchés d'une hauteur suffisamment grande. Plus précisément, il doit y avoir un étage f dans un immeuble suffisamment haut tel qu'un œuf lâché du f -ième étage se casse, mais un lâché du $(f - 1)$ -ième étage ne se cassera pas. Si l'œuf se casse toujours, alors $f = 1$. Si l'œuf ne se casse jamais, alors $f = n + 1$.

On cherche à trouver l'étage critique f en utilisant un immeuble de n étages. La seule opération que l'on peut effectuer est de lâcher un œuf d'un étage et voir ce qui se passe. On commence avec k œufs, et on cherche à lâcher des œufs aussi peu de fois que possible. Les œufs cassés ne peuvent pas être réutilisés. Soit $E(k, n)$ le nombre minimum de lâchers d'œufs qui suffiront toujours.

- 1) Montrer que $E(1, n) = n$.
- 2) Montrer que $E(k, n) = O(kn^{1/k})$.
- 3) Trouver une relation de récurrence pour $E(k, n)$. Quel est la complexité de l'algorithme de programmation dynamique pour trouver $E(k, n)$?